

Hands-On Activity

CNN-LSTM Hybrid Models and Hyperparameter Optimization

Companion activity for 04_cnn_lstm_energy_forecasting.ipynb and 05_cnn_lstm_optimized_energy_forecasting.ipynb (extends the RNN/LSTM/GRU set, same dataset and task)

Estimated time: 140 minutes **Prerequisite:** a completed run of 04_cnn_lstm_energy_forecasting.ipynb and 05_cnn_lstm_optimized_energy_forecasting.ipynb (and, ideally, the earlier RNN/LSTM/GRU set for Part D).

Learning Objectives

- Explain why combining Conv1D and LSTM layers can help a forecasting model, and what causal padding does in a temporal Conv1D layer.
- Build and interpret a CNN-LSTM Sequential model's shape flow (Input → Conv1D → MaxPooling1D → LSTM → Dense).
- Define a Keras Tuner search space and run a random search over CNN-LSTM hyperparameters.
- Compare a hand-picked baseline against a hyperparameter-tuned model using validation loss and parameter count, and connect search results back to which hyperparameters mattered most.
- Distinguish MISO (multivariate input, univariate output) from MIMO (multivariate input, multivariate output) configurations, and modify a Sequential model's target construction and final Dense layer to switch between them.
- Place the plain LSTM, baseline CNN-LSTM, and tuned CNN-LSTM results into one coherent comparison on the same forecasting task.

Materials

- 01_vanilla_rnn_energy_forecasting.ipynb, 02_lstm_energy_forecasting.ipynb, 03_gru_energy_forecasting.ipynb (for the Part D comparison).
- 04_cnn_lstm_energy_forecasting.ipynb and 05_cnn_lstm_optimized_energy_forecasting.ipynb (this activity's primary notebooks).
- A Python environment with TensorFlow/Keras, keras-tuner, pandas, matplotlib, pydot, and Graphviz.
- Optional: the real Kaggle dataset. Works either way — note in your answers which data source you used.

Part A — Conceptual Warm-Up (individual, ~15 min)

Answer briefly (2–4 sentences each) before touching any code.

A1.

Notebook 04's Conv1D layer uses padding="causal". Explain what this padding choice does to which timesteps each Conv1D output position can "see", and why this matters more for architectures that produce one prediction per timestep (like the synced many-to-many notebook) than it does for this specific many-to-one setup, where the label is already outside the whole window.

Answer:

With padding="causal", Keras adds zeros only to the left (past) side of the sequence, so the Conv1D output at timestep t only uses inputs from $t-2$, $t-1$ and t (with `KERNEL_SIZE = 3`). It never sees $t+1$ or later, and the output length stays at 24. This matters most in a synced many-to-many model, where every timestep has its own label, often the next hour's value. There, a non-causal filter at position t could look at $t+1$, which is the answer it is supposed to predict.

CNN-LSTM pipeline: Conv1D extracts local patterns, LSTM models longer-range dependencies



Figure A1. The CNN-LSTM pipeline in `04_cnn_lstm_energy_forecasting.ipynb`: shapes shown are for the default `FILTERS=32`, `WINDOW=24`, `UNITS=32`.

A2.

`MaxPooling1D(pool_size=2)` shrinks the sequence from 24 timesteps to 12 before the LSTM sees it. What information is discarded by pooling, and why might discarding it still help the LSTM that follows?

Answer:

`MaxPooling1D(pool_size=2)` looks at each non-overlapping pair of hours (1 and 2, 3 and 4, and so on). For each of the 32 `Conv1D` filters, it keeps only the larger activation. What gets discarded is the weaker value in each pair and the exact timing, meaning which of the two hours the pattern actually peaked in. That can still help the LSTM: it now reads 12 steps instead of 24, so gradients travel through half as many recurrent steps and learning is faster and more stable. It also receives a cleaner summary of "the strongest pattern detected in each 2-hour block", which filters out small noise and acts as a light regularizer.

A3.

In notebook 05, `build_model(hp)` calls `model.compile(...)` inside the function itself, and the notebook later uses `fit_only()` rather than `compile_and_fit()` to train the best model. Why is this necessary here, when every earlier notebook used `compile_and_fit()` without issue?

Answer:

In Notebook 05 the learning rate is one of the hyperparameters being searched, and the optimizer only receives its learning rate during `model.compile()`. So compiling has to happen inside `build_model(hp)`, where the sampled `hp` value exists. Keras Tuner also expects `build_model` to return a ready-to-train model, because it calls `fit()` itself on every trial. `tuner.hypermodel.build(best_hp)` therefore returns a model already compiled with the tuned learning rate. Calling `compile_and_fit()` on it would recompile with `optimizer="adam"`, which resets the learning rate to Adam's default of 0.001 and quietly throws away part of what the search found.

A4.

In one or two sentences each, distinguish grid search, random search, and Hyperband/Bayesian search as hyperparameter search strategies.

- A. **Grid search** tries every combination of the hyperparameter values you list. It is thorough and easy to reproduce, but the number of trials multiplies with each new hyperparameter, so it quickly becomes too expensive.
- B. **Random search** samples a fixed number of combinations at random from the search space. For the same budget, it usually finds good settings faster than grid search, because only a few hyperparameters tend to matter and random sampling tries more distinct values of each one.
- C. **Hyperband and Bayesian optimization** are "smarter" strategies that use results as they come in. Hyperband starts many configurations with only a few epochs each, then keeps training only the best ones because of the early stopping of the weak ones. Bayesian optimization builds a statistical model of "hyperparameters → val_loss" from past trials and uses it to pick the most promising next trial.

Part B — Hands-On Modification Tasks (pair or individual, ~45 min)

Do each task in the notebook named. Re-run from the point of change onward, then record what changed using the printed “ran X/100 epochs ... params=...” line (or the tuner's results_summary for B3–B5).

Task B1 — Deepen the conv stack (Notebook 04)

Uncomment the second Conv1D + MaxPooling1D block in the baseline model.

Record: new parameter count, new val_loss/val_mae.

```
Parameter Count: 8,865
val_loss: 0.00085
val_mae: 0.02114
```

Reflect: Did a deeper conv stack help on this dataset size, or was one Conv1D block enough?

Model	Params	val_loss	val_mae (scaled)
Baseline (1 conv block)	8,865	0.00085	0.02114
Deeper (2 conv blocks)	11,969	0.00091	0.02145
Change	+3,104 (+35%)	+0.00006 (about 7% worse)	+0.00031 (about 1.5% worse)

Based on the values in the change row, a deeper conv stack did not help on this dataset.

Task B2 — Change kernel and pool size (Notebook 04)

Change KERNEL_SIZE from 3 to 5, and POOL_SIZE from 2 to 3. Re-run and note the new shapes at each pipeline stage (update the draw_pipeline() call's shape labels to match, or just compute them by hand).

Record: the new post-pooling sequence length, and the new val_loss.

```
Post-pooling length: 8
new val_loss: 0.00087
```

Task B3 — Widen the search (Notebook 05)

Increase MAX_TRIALS from 8 to 20 and SEARCH_EPOCHS from 20 to 30. Re-run the search cell.

Record: the new best hyperparameters and best val_loss, and how much longer the search took.

```
...
      params  final_val_loss_mse  final_val_mae
model
baseline_fixed_hp    8865          0.000822      0.020560
tuned                10929          0.000893      0.021297
```

Reflect: Was the improvement worth the extra compute time?

The tuned model is bigger and slightly less accurate, so the hand-picked baseline wins on both accuracy and efficiency. The tuned model is 23% bigger but 8.6% worse on MSE and 3.6% worse on MAE.

Task B4 — Try a different search strategy (Notebook 05)

Replace `kt.RandomSearch(...)` with `kt.Hyperband(build_model, objective="val_loss", max_epochs=SEARCH_EPOCHS, factor=3, directory="kt_search", project_name="cnn_lstm_hyperband", overwrite=True, seed=42)`, then call `tuner.search(...)` the same way.

Record: Hyperband's best `val_loss` and total search time, compared to RandomSearch's.

	RandomSearch (B3)	Hyperband (B4)
Best val_loss	0.00090 (0.000901)	0.00090 (0.000898)
Total search time	53 min	about 60 min
Trials	20 (each 30 epochs)	90 (most 1 to 10 epochs, a few up to 30)

Task B5 — Add a hyperparameter (Notebook 05)

Add one more tunable hyperparameter to `build_model(hp)` — for example an L2 regularization strength on the Dense layer, or a second Dense layer's width. Re-run the search.

Note: Highlighted test are the added codes

```
def build_model(hp):
    filters = hp.Int("filters", min_value=16, max_value=64, step=16)
    kernel_size = hp.Choice("kernel_size", [2, 3, 5])
    pool_size = hp.Choice("pool_size", [2, 3])
    lstm_units = hp.Int("lstm_units", min_value=16, max_value=96, step=16)
    dropout_rate = hp.Float("dropout_rate", min_value=0.0, max_value=0.4, step=0.1)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2, sampling="log")
    l2_strength = hp.Choice("l2_strength", [0.0, 1e-5, 1e-4, 1e-3])

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(WINDOW, N_FEATURES)),
        tf.keras.layers.Conv1D(filters=filters, kernel_size=kernel_size, activation="relu", padding="causal"),
        tf.keras.layers.MaxPooling1D(pool_size=pool_size),
        tf.keras.layers.LSTM(lstm_units, return_sequences=False),
        tf.keras.layers.Dropout(dropout_rate),
        tf.keras.layers.Dense(1, kernel_regularizer=tf.keras.regularizers.l2(l2_strength)),
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mse",
        metrics=["mae"],
    )
    return model
```

Record: whether your new hyperparameter appears with a consistent value across the top 3 trials in `tuner.results_summary(num_trials=3)`.

Results summary

Results in kt_search/cnn_lstm_hyperband

Showing 3 best trials

Objective(name="val_loss", direction="min")

Trial 0072 summary

Hyperparameters:

filters: 32

kernel_size: 2

pool_size: 2

lstm_units: 32

dropout_rate: 0.1

learning_rate: 0.0022108968049675453

l2_strength: 0.0

tuner/epochs: 30

tuner/initial_epoch: 10

tuner/bracket: 2

tuner/round: 2

tuner/trial_id: 0069

Score: 0.0009107955847866833

Trial 0082 summary

Hyperparameters:

filters: 64

kernel_size: 3

pool_size: 2

lstm_units: 80

dropout_rate: 0.30000000000000004

learning_rate: 0.0013638404825740366

l2_strength: 1e-05

tuner/epochs: 30

tuner/initial_epoch: 10

tuner/bracket: 1

tuner/round: 1

tuner/trial_id: 0074

Score: 0.0009286955464631319

Trial 0073 summary

Hyperparameters:

filters: 32

kernel_size: 2

pool_size: 2

lstm_units: 80

dropout_rate: 0.0

learning_rate: 0.0007781436518320102

l2_strength: 1e-05

tuner/epochs: 30

tuner/initial_epoch: 10

tuner/bracket: 2

tuner/round: 2

tuner/trial_id: 0071

Score: 0.0009396542445756495

	Trial 0072 (best)	Trial 0082	Trial 0073
l2_strength	0.0	1e-5	1e-5
filters	32	64	32
kernel_size	2	3	2
pool_size	2	2	2
lstm_units	32	80	80
dropout_rate	0.1	0.3	0.0
learning_rate	0.0022	0.0014	0.0008
Score (val_loss)	0.00091	0.00093	0.00094

The new hyperparameter did not improve the result. The best val_loss with L2 in the new search was 0.00091, compared with 0.00090 in B4 without it.

Part C — Extending to Multivariate Input and Output (individual or pair, ~50 min)

Every model built so far in Notebooks 04 and 05 takes multivariate input (the 5-column FEATURE_COLS window) and produces univariate output (a single predicted price). This input/output combination is sometimes labeled MISO (Multiple Inputs, Single Output). This part has you build the MIMO (Multiple Inputs, Multiple Outputs) counterpart: the same 5-feature input window, but predicting two targets at once, still on the same 24-hour many-to-one task.

C1 — Name the baseline (conceptual, ~5 min)

Label the baseline CNN-LSTM model from Notebook 04 using the MISO/MIMO terminology, and state precisely what “multivariate” refers to on the input side (FEATURE_COLS) versus what would have to change on the output side to make the model MIMO instead of MISO.

Answer:

The baseline CNN-LSTM from Notebook 04 is a **MISO (Multiple Inputs, Single Output)** model. On the input side, "multivariate" means that every timestep in the 24-hour window carries **five features** from FEATURE_COLS: total load, solar generation, onshore wind generation, fossil gas generation, and price. So each input sample has the shape **(24, 5)**, and the full input is X with shape **(n, 24, 5)**. On the output side, the model predicts only **one value**, the next hour's "price actual", so y has shape **(n, 1)** and the last layer is Dense(1). To make it **MIMO**, two things must change: the **target construction** must build each label from several columns (for example, price and load, giving y shape **(n, 2)**), and the final layer must output one unit per target (Dense(2)).

C2 — Build a MIMO variant (hands-on, ~30 min)

In a copy of Notebook 04's baseline model, build a MIMO version that predicts two targets at once instead of one:

1. Define `TARGET_COLS = ["price actual", "total load actual"]` alongside the existing single `TARGET_COL`.
2. Write a `make_many_to_one_multi(data, window=24)` that mirrors `make_many_to_one`, but builds `y` from both target columns: `y.append(data[TARGET_COLS].values[t + window])` so each label is a 2-vector instead of a scalar.
3. Change the model's final layer from `tf.keras.layers.Dense(1)` to `tf.keras.layers.Dense(2)`.
4. Retrain for the same `EPOCHS = 100` and the same `WINDOW`, `FILTERS`, `KERNEL_SIZE`, `POOL_SIZE`, `UNITS` as the baseline.

Record: the new `y` shape, the new Dense layer's parameter count, and the combined `val_loss` Keras reports (it averages MSE across both output units by default).

Item	MISO baseline	MIMO (C2)
Targets	price only	price + total load
X shape (train)	(24520, 24, 5)	(24520, 24, 5), unchanged
y shape (train)	(24520, 1)	(24520, 2)
Final layer	Dense(1)	Dense(2)
Dense params	33	66 (32 × 2 weights + 2 biases)
Total params	8,865	8,898 (+33)
Combined val_loss	0.00085 (price only)	0.00074 (average of price and load MSE)
val_mae (scaled)	0.02114 (price only)	0.01913 (average of both)
Prediction shape (val)	(5236, 1)	(5236, 2)

C3 — Per-target accuracy (hands-on/analysis, ~15 min)

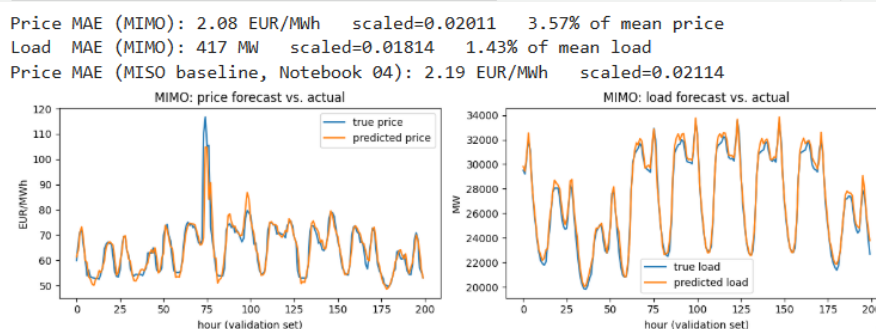
The combined `val_loss` from C2 mixes both targets' errors together, and price and load live on very different numeric scales, so it does not tell you how well either target is predicted on its own. Inverse-transform each output column separately using that column's own scaler range, not `TARGET_IDX` alone — for example:

```
price_min, price_max = scaler.data_min_[FEATURE_COLS.index("price actual")],
scaler.data_max_[FEATURE_COLS.index("price actual")]

load_min, load_max = scaler.data_min_[FEATURE_COLS.index("total load actual")],
scaler.data_max_[FEATURE_COLS.index("total load actual")]
```

Then compute a separate MAE in real units (EUR/MWh for price, MW for load) for each of the two outputs.

Record: the per-target MAE for price and for load.



Model	Price MAE	Scaled
MISO baseline (Notebook 04)	2.19 EUR/MWh	0.02114
MIMO (C2)	2.08 EUR/MWh	0.02011
Change	-0.11 EUR/MWh (about 5% better)	

Reflect: Which target is predicted more accurately? Does training on both jointly seem to help or hurt the price prediction compared to the single-target (MISO) baseline's val_mae from Notebook 04?

The Load is accurately predicted. Its MAE of 417 MW is only 1.43% of the average load, while the price MAE of 2.08 EUR/MWh is 3.57% of the average price, so load is predicted about 2.5 times more accurately in relative terms. This makes sense: electricity demand follows a very regular daily and weekly routine, while price reacts to fuel costs, renewable output and market events. The plots show this clearly: the load forecast almost overlaps the true curve, while the price forecast still misses the sudden spike near hour 75.

Joint training appears to help slightly. The MIMO model's price MAE (2.08 EUR/MWh, scaled 0.02011) is about 5% lower than the MISO baseline's (2.19 EUR/MWh, scaled 0.02114), with only 33 extra parameters.

C4 — Reflection.

Does moving from MISO to MIMO change which typology diagram (one-to-one / many-to-one / one-to-many / many-to-many) applies to this architecture? Why or why not?

No. The model is still many-to-one. The typology describe the time axis: how many timesteps go in and how many timesteps come out. MISO and MIMO describe the feature axis: how many variables are predicted at each output step. Moving to MIMO only made the single output step wider. The model still reads 24 hourly timesteps and produces one prediction for one future hour (hour 25); that prediction is now a 2-value vector (price and load) instead of a single number. In shape terms, y changed from $(n, 1)$ to $(n, 2)$, not to $(n, T, 2)$, so there is still exactly one output timestep.

Part D — Cross-Model Comparison (individual or group, ~30 min)

Using the printed metrics from 02_lstm_energy_forecasting.ipynb (many_to_one), 04_cnn_lstm_energy_forecasting.ipynb (baseline), and 05_cnn_lstm_optimized_energy_forecasting.ipynb (tuned), fill in the table below. (The MIMO variant from Part C is left out of this table on purpose — see the note below it.)

Model	Params	val_loss	val_mae
Plain LSTM (many-to-one, Notebook 02)	4,897	0.00077	0.01930
CNN-LSTM baseline (Notebook 04)	8,865	0.00085	0.02114
CNN-LSTM tuned (Notebook 05)	10,929	0.00089	0.02130

Note: the MIMO model's val_loss from Part C is an average over two targets on different scales, so it is not on the same footing as the single-target val_loss values above — that is a real property of multi-output regression, not a mistake in your table.

Reflection questions

D1.

Between the plain LSTM and the CNN-LSTM baseline, which had the lower val_loss on your run? What does that suggest about how much position-independent local structure this dataset actually has for a convolution to exploit?

Answer:

On my run, the **plain LSTM had the lower val_loss**: 0.00077, compared with 0.00085 for the CNN-LSTM baseline, about 10% better, while using only 4,897 parameters versus 8,865. This suggests that this dataset has **little position-independent local structure** for a convolution to exploit. Hourly electricity prices are dominated by a smooth, repeating daily cycle and by strong dependence on the most recent hours, not by short, sharp shapes (such as a ramp or spike) that can appear anywhere in the window. The LSTM already captures that pattern by reading the 24 hours one step at a time at full resolution. The Conv1D and MaxPooling1D stage instead summarizes pairs of hours and throws away exact timing, which for a next-hour forecast removes useful detail.

D2.

Did hyperparameter tuning close the gap (or extend the lead) between the CNN-LSTM and the plain LSTM? State the direction and size of the change.

Answer:

The gap got wider, not smaller. Hyperparameter tuning did not close the gap. It slightly extended the plain LSTM's lead. The val_loss gap grew from 0.00008 to 0.00012, an increase of 0.00004. In other words, the plain LSTM's lead went from about 10% to about 16%.

D3.

Looking at your tuner's `results_summary()` across its top 3 trials, which single hyperparameter appeared most consistently favorable (e.g. did most top trials prefer a particular kernel size, a small or large dropout rate)?

Answer:

The learning rate was the most consistently favorable hyperparameter. All three top trials in my RandomSearch chose values in a narrow band between about 0.0011 and 0.0017, even though the search space covered a 100-fold range .

Deliverables

5. Your modified notebook(s), or a diff/screenshot, for each Part B task.
6. Your MIMO notebook variant from Part C, with the per-target MAE results.
7. Written answers to Parts A, C, and D.
8. The completed comparison table from Part D.